SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192525074
Student Name	Shaik Ayesha Tabassum

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
   FOR j ← i + 1 TO n − 1 DO
      IF Product[i].title = Product[j].title AND Product[i].seller =
Product[j].seller THEN
         MarkDuplicate(j)
      END IF
   END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
   FOR j ← 0 TO i − 1 DO
      Audit(Request[j])          // basic operation
   END FOR
   i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by

comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies T(n) = T(n/2) + 1, T(1) = 1.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: T(n) = 4T(n/2) + n. A proposed redesign removes the replication, giving T(n) = 2T(n/2) + n.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving T(n) = 2T(n/2) + n log n. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's Θ(n log n) claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to T(n) = 2T(n/2) + n log n; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1 , c_2 and n_0 , and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I **Shaik Ayesha Tabassum** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **Shaik Ayesha Tabassum** Date: 11/10/2026

(a) Understand the Problem

Question: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how its execution count grows as (n) increases. Justify each choice briefly.

Answer:

1. M1 – Duplicate-Listing Detector: The input size is (n), the number of products. The basic operation is comparing product titles and sellers using nested loops. It compares every pair of products, so the number of comparisons is frac{n(n-1)}{2}. Its time complexity is (Theta(n^2)).

2. M2 – API Rate Limiter: The input size is (n), the number of requests. The basic operation is the Audit() call inside a loop that doubles its counter each iteration. The total number of calls grows linearly with (n), so its time complexity is (Theta(n)).

3. M3 – Product Search:

- S1 – Sequential Search: Checks products one by one until the required product is found. Its time complexity is (O(n)).
- S2 – Binary Search: Searches a sorted product index by checking the middle entry and eliminating half the remaining entries after each comparison. Its time complexity is (Theta(log n)).

4. M4 – Recommendation Engine: The current design divides the data into four sub problems of size (n/2) and combines their results in linear time. Its recurrence is (T(n)=4T(n/2)+n), giving (Theta(n^2)). The proposed redesign uses two subproblems, (T(n)=2T(n/2)+n), giving (Theta(n log n)).

5. M5 – Sales Analytics Engine: The engine divides the data into two halves and sorts records by revenue at each level. Its recurrence is (T(n)=2T(n/2)+n log n), giving (Theta(n log^2 n)). The

separate report generator performs (f(n)=3n^2+10n log_2 n+50) operations, so its time complexity is (Theta(n^2)).

(b) Analyse the Iterative Modules

Question: For M1, derive the exact number of comparisons as a function of (n) and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the (Theta(n log n)) claim is correct, and verify the result for (n=1,024).

Answer:

M1 – Duplicate-Listing Detector

The outer loop runs from i=0 to (n-2). For each (i), the inner loop compares the current product with all following products.

The total number of comparisons is

$$C(n)=(n-1)+(n-2)+\ldots\ldots\ldots+1$$

Using the sum of the first (n-1) positive integers,

$$C(n)= frac\{n(n-1)\}\{2\}$$

Therefore:

- Time complexity: (Theta(n^2))
- Auxiliary space complexity: (Theta(1)), because the algorithm uses only a fixed amount of extra memory.

M2 – API Rate Limiter

The outer loop counter doubles each time: (1,2,4,8,…..). The inner loop executes once for every value of the counter.

Let (k=log_2 n). The total number of Audit operations is

$$A(n)=1+2+4+\ldots\ldots+2^{k-1}$$

Using the geometric series formula,

$$\{A(n)=2^k-1=2^{\log_2 n}-1\}$$

Since this count grows proportionally to (n), the tight complexity is

$$\text{Theta}(n)$$

Therefore, the team member's claim of (Theta(n log n)) is not correct; it is only a loose upper bound.

$$\text{For } n=1{,}024=2^{10},$$

$$A(1024)=2^{10}-1=\{1{,}023\}$$

Result: M1 has quadratic time complexity, while M2 has linear time complexity.

(c) Solve a Recurrence by Substitution

Question: Solve the recurrence for S2 using backward substitution, show every intermediate step, derive the exact closed form, and state the time and space complexities of iterative and recursive implementations.

Answer:

For binary search,

$$T(n)=T(n/2)+1, T(1)=1$$

Using backward substitution:

$$T(n)=T(n/2)+1$$

Substitute (T(n/2)=T(n/4)+1):

$$T(n)=T(n/4)+2$$

Substitute again:

$$T(n)=T(n/8)+3$$

Continuing this process, after (k) substitutions,

$$T(n)=T(n/2^k)+k$$

The process stops when (n/2^k=1). Therefore,

$$frac{n}{2^k}=1$$

$$n=2^k->k=log_2 n$$

Since (T(1)=1),

$$T(n)=1+ log_2 n$$

Thus, for (n) that is a power of two,

$${T(n)=1+ log_2 n}$$

For general positive integer input sizes, the exact count under this recurrence convention is (T(n)=log_2 +1).

Complexity comparison:

- Iterative binary search: Time (Theta(log n)), auxiliary space (Theta(1)).
- Recursive binary search: Time (Theta(log n)), auxiliary space (Theta(log n)) because of the recursive call stack.

Result: Both implementations have logarithmic time complexity, but iterative binary search uses less auxiliary space.

(d) Apply the Master Theorem

Question: Apply the Master Theorem to M4's current design and proposed redesign. Identify (a), (b), (f(n)), (n^{log_b a}), and the case used. Explain the effect of doubling the input size. For M5, determine whether a standard case applies and derive its complexity using the extended Master Theorem or a recursion tree.

Answer:

The Master Theorem is applied to recurrences of the form

T(n)=aT(n/b)+f(n)

Here, (a) is the number of sub problems, (b) is the division factor, and (f(n)) is the cost of combining the results.

M4 – Current design

Given:

T(n)=4T(n/2)+n

- (a=4), (b=2), and (f(n)=n).
- (n^{log_b a}=n^{log_2 4}=n^2).
- Since (f(n)=n) grows slower than (n^2), Master Theorem Case 1 applies.

Therefore,

{T(n)=\Theta(n^2)}

When the input doubles, the running time approaches four times the previous running time for large (n).

M4 – Proposed redesign

Given:

T(n)=2T(n/2)+n

- (a=2), (b=2), and (f(n)=n).
- (n^{\og_b a}=n^{log_2 2}=n).
- Since (f(n)=Theta(n)), Master Theorem Case 2 applies.

Therefore,

{T(n)= Theta(n log n)}

When the input doubles, the running time increases by approximately a factor of two for large (n), with a small additional logarithmic effect.

M5 – Sales Analytics Engine

Given:

T(n)=2T(n/2)+n log n

Here, (a=2), (b=2), and (n^{log_b a}=n). The cost (n log n) is larger than (n) by a logarithmic factor, so none of the three basic Master Theorem cases directly gives the result.

Using the extended Master Theorem,

T(n)=Theta(n log^{1+1}n)

Therefore,

{T(n)=Theta(n log^2 n)}

Result: The M4 redesign improves the complexity from (Theta(n^2)) to (Theta(n log n)), while M5 has complexity (Theta(n log^2 n)).

(e) Analyse Best, Average and Worst Cases

Question: For S1 and S2, identify the input condition that produces the best, average, and worst cases and give the number of comparisons. For S1, derive the average number of comparisons when the probability of finding the product is (p), and evaluate it for (n=1,000,000\) and (p=0.5).

Answer:

S1 – Sequential Search

Sequential search examines elements one by one.

- Best case: The product is the first element. Comparisons = (1).
- Average case: The product is equally likely to occur at any position, or it may be absent. The expected count depends on the probability (p).
- Worst case: The product is the last element or is absent. Comparisons = (n).

If the product exists, its average position requires ((n+1)/2) comparisons. If it does not exist, (n) comparisons are required.

Therefore, the average number of comparisons is

$$A(n)=p \ frac{n+1}{2}+(1-p)n$$

Simplifying,

$$\{A(n)= frac\{n(2-p)+p\}\{2\}\}$$

For (n=1,000,000) and (p=0.5),

$$A(n)= frac\{1,000,000(2-0.5)+0.5\}\{2\}$$

$$A(n)=750,000.25 \text{ Comparisions}$$

S2 – Binary Search

Binary search requires the product index to be sorted.

- Best case: The product is at the middle position. Comparisons = (1).
- Average case: The product is found after repeatedly halving the search range. Under the assignment's approximation, comparisons are about (log_2 n-1).
- Worst case: The search continues until the range is reduced to one element or the product is confirmed absent. Comparisons are at most ([log_2 n]+1).

For (n=1,000,000),

$$\log_2(1{,}000{,}000)\sim 19.93$$

The worst-case comparison count is

$$[19.93]+1=20$$

Result: Binary search is much more efficient than sequential search for a large, sorted product catalogue.

(f) Prove an Asymptotic Bound

Question: Using formal definitions, prove that the report generator's cost ($f(n)=3n^2+10n \log_2 n+50$) is ($\Theta(n^2)$) by finding constants ($c_1,c_2,n_0$), and show that ($f(n)$) is not ($O(n)$).

Answer:

The given function is

$$f(n)=3n^2+10n \log_2 n+50$$

To prove that ($f(n)=\Theta(n^2)$), we must find positive constants (c_1,c_2,n_0) such that

$$c_1n^2 <= f(n) <= c_2n^2$$

for all $(n >= n_0)$.

Lower bound

Since $(10n \log_2 n)$ and (50) are non-negative for $(n>=1)$,

$$f(n) >= 3n^2$$

Choose

$$c_1 = 3$$

Upper bound

For $(n>=1)$, $(\log_2 n <= n)$. Also, $(50 <= 50n^2)$. Therefore,

$$10n \log_2 n <= 10n^2$$

Combining these inequalities,

$$f(n) <= 3n^2 + 10n^2 + 50n^2$$

$$f(n) <= 63n^2$$

Choose $(c_2=63)$.

Thus, we can choose

$$\{c_1=3, c_2=63, n_0=1\}$$

Therefore,

$$3n^2 <= f(n) <= 63n^2$$

This proves that

{f(n)=Theta(n^2)}

Show that (f(n)) is not (O(n))

For (f(n)) to be (O(n)), there must be a constant (c>0) such that (f(n) cn) for sufficiently large (n).

However,

frac{f(n)}{n}>=frac{3n^2}{n}=3n]

As (n) increases, (3n) grows without bound. Therefore, no fixed constant (c) can satisfy the required upper bound.

Hence,

f(n)!=O(n)

Result: The report generator has quadratic time complexity, not linear time complexity.

(g) Scalability Decision

Question: Estimate the basic operations and running time of M1, M2, S2, M4 (both designs), and M5 for (n=1,000,000). Identify the modules that fail the 10-minute nightly processing window and recommend which modules must be redesigned and how.

Answer:

The question specifies a processing speed of (10^8) operations per second and a nightly processing window of 10 minutes.

Maximum operations=10^8*600

60,000,000,000 operations

The following estimates use the operation counts and cost formulas given in the assignment.

Module	Complexity	Estimated operations	Estimated time
M1 – Duplicate detector	(Theta(n^2))	499,999,500,000	5,000 s
M2 – Rate limiter	(Theta(n))	1,048,575	0.0105 s
S2 – Binary search	(Theta(log n))	20	Less than 0.001 s
M4 – Current design	(Theta(n^2)	1,999,999,000,000	20,000 s
M4 – Redesign	(Theta(nlog n))	20,931,569	0.209 s
M5 – Analytics engine	(Theta(nlog^2 n))	209,599,497	2.10 s
M5 – Report generator	(Theta(n^2))	3,000,199,315,736	30,002 s

Note: Times are estimates obtained by dividing the operation counts by \(10^8\).

Modules that fail

Three components exceed the 600-second limit:

1. M1 – Duplicate detector: Requires approximately 83.3 minutes.
2. M4 – Current recommendation engine: Requires approximately 5.56 hours.
3. M5 – Report generator: Requires approximately 8.33 hours.

M2, S2, the redesigned M4, and the M5 analytics engine meet the time limit.

Recommended redesigns

1. M1 – Duplicate detector

Replace the nested-loop comparison with a hash table keyed by the pair (title, seller). Each product is checked against the set and inserted if it is new.

- Expected time complexity: (Theta(n))
- Auxiliary space complexity: (Theta(n))

This avoids comparing every pair of products.

2. M4 – Recommendation engine

Adopt the proposed redesign:

T(n)=4T(n/2)+n -> T(n)=2T(n/2)+n

This reduces the complexity from (Theta(n^2)) to (Theta(n log n)), bringing the estimated processing time down to about 0.21 seconds.

3. M5 – Report generator

The report generator has quadratic complexity because its dominant term is (3n^2). Its implementation must be redesigned to avoid repeatedly processing all pairs of records. Possible approaches include indexing, aggregation, and grouping records before report generation. The exact improvement depends on the report's requirements.